# SystemVerilog for Design and Verification

***Engineer Explorer Series***

**Week 12 Lab Document**

**Lab Manual** **Revision 1.0**

cadence®

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

# Table of Contents

## SystemVerilog for Design and Verification

## Overview of Labs

In these labs, you use SystemVerilog language constructs to complete simple, common design tasks. You can use Verilog-2001 constructs to complete some of the design tasks described here, but that would obviously defeat the purpose of the labs. Where possible, try to use SystemVerilog constructs.

This lab book assumes you are familiar with the Cadence® simulator, which you use in this course. If this is not the case, please ask your instructor or contact Cadence for information on running the simulator.

The goal is not to complete all the exercises during the course but to learn from each exercise at your own pace.

## Software Releases

These exercises and code examples have been written and tested on the following releases:

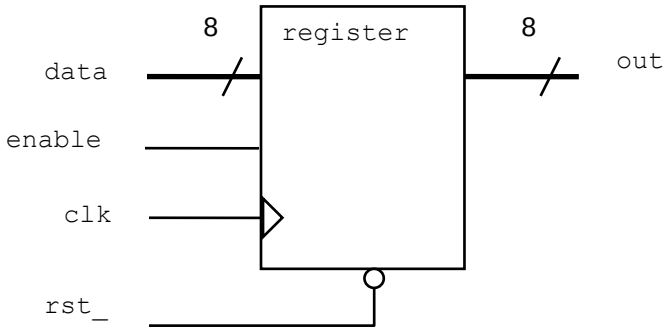
- Xcelium™ 24.09

# Lab 1 Modeling a Simple Register

**Objective: To use SystemVerilog procedural constructs to model a simple register.**

Create a simple register design using SystemVerilog and Verilog-2001 constructs and test it using the supplied testbench.

Read the specification first and then follow the instructions in the Creating the Register Design section.




## Specification

- `data` and `out` are both 8-bit `logic` vectors.
- `rst_` is asynchronous and active low.
- The register is clocked on the rising edge of `clk`.
- If `enable` is high, the input `data` is passed to the output `out`.
- Otherwise, the current value of `out` is retained in the register.

## Creating the Register Design

Work in the `lab01-reg` directory:

1. Create a new file called `register.sv`, containing a module named `register`.

1. Write the register model using the following SystemVerilog constructs:

   a. `always_ff` procedural block

   b. `timeunit` and `timeprecision`

c. Verilog2001 ANSI-C port declarations

## Testing the Register Design

1. A testbench is provided in the file `register_test.sv`. Simulate the testbench and register design.

   You should see the following results:

```
time=   0.0 ns enable=x rst_=1 data=xx out=xx
time=  15.0 ns enable=x rst_=0 data=xx out=00
time=  25.0 ns enable=0 rst_=1 data=xx out=00
time=  35.0 ns enable=1 rst_=1 data=aa out=aa
time=  45.0 ns enable=0 rst_=1 data=55 out=aa
time=  55.0 ns enable=x rst_=0 data=xx out=00
time=  65.0 ns enable=0 rst_=1 data=xx out=00
time=  75.0 ns enable=1 rst_=1 data=55 out=55
time=  85.0 ns enable=0 rst_=1 data=aa out=55
REGISTER TEST PASSED
```

   Debug your register as required.

2. When the test passes, copy the `register.sv` file into the `../sv_src` directory. You will use it later for the complete VeriRISC design lab.

## Run Command

xrun register.sv register_test.sv ( Batch Mode)

xrun register.sv register_test.sv -gui -access +rwc ( GUI Mode)

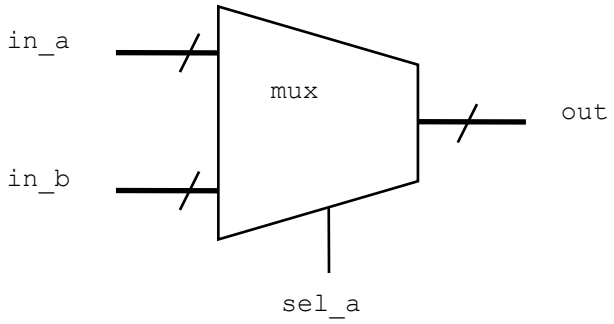
# Lab 2 Modeling a Simple Multiplexor (Optional)

**Objective: To use SystemVerilog procedural constructs to model a simple multiplexor.**

---

Create a simple multiplexor design using SystemVerilog and Verilog-2001 constructs and test it using the supplied testbench.

Read the specification first and then follow the instructions in *Creating the MUX Design*.




## Specification

- `in_a`, `in_b` and `out` are all `logic` vectors.
- The MUX width is parameterized with a default value of 1.
- If `sel_a` is `1'b1`, input `in_a` is passed to the output.
- If `sel_a` is `1'b0`, input `in_b` is passed to the output.

## Creating the MUX Design

Working in the `lab02-mux` directory, perform the following.

1. Create a new file called `scale_mux.sv`, containing a module named `scale_mux`.

1. Write the MUX model using the following SystemVerilog and Verilog constructs:
   - Verilog2001 ANSI-C port declarations
     - Parameterize the MUX width and give it a default value of 1
   - `always_comb` procedural block
   - timeunit and timeprecision

- `unique case` construct
  - Include a `default` match that sets the output to unknown

## Testing the MUX Design

1. A testbench is provided in the `scale_mux_test.sv` file. Simulate the testbench and MUX design.

   You should see the following results:

   ```
   0ns in_a=00 in_b=00 sel_a=0 out=00
   1ns in_a=00 in_b=00 sel_a=1 out=00
   2ns in_a=00 in_b=ff sel_a=0 out=ff
   3ns in_a=00 in_b=ff sel_a=1 out=00
   4ns in_a=ff in_b=00 sel_a=0 out=00
   5ns in_a=ff in_b=00 sel_a=1 out=ff
   6ns in_a=ff in_b=ff sel_a=0 out=ff
   7ns in_a=ff in_b=ff sel_a=1 out=ff
   MUX TEST PASSED
   ```

   Debug your MUX as required.

2. When the test passes, copy the `scale_mux.sv` file into the `../sv_src` directory. You will use it later for the complete VeriRISC design lab.

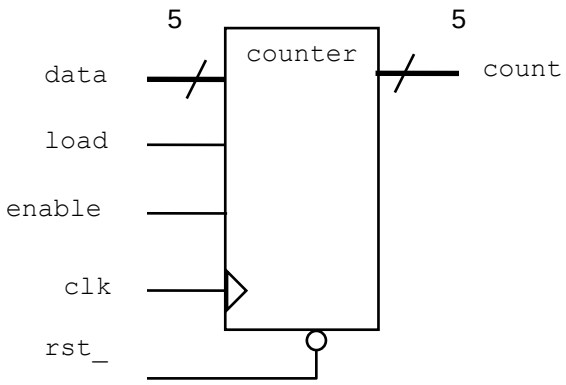
# Lab 3 Modeling a Simple Counter

**Objective: To use SystemVerilog procedural constructs and operators to model a simple counter.**

---

Create a simple loadable, enabled counter design using SystemVerilog and Verilog-2001 constructs and test it using the supplied testbench.

Read the specification first and then follow the instructions in the Creating the Counter Design section in this lab.




## Specification

- `data` and `count` are both 5-bit `logic` vectors.
- `rst_` is asynchronous and active low.
- The counter is clocked on the rising edge of `clk`.
  - If `load` is high, the counter is loaded from the input `data`.
  - Otherwise, if `enable` is high, `count` is incremented.
  - Otherwise, `count` is unchanged.

## Creating the Counter Design

Working in the `lab03-count` directory, perform the following.

1. Create a new file called `counter.sv`, containing a module named `counter`.

3. Write the counter model using the following SystemVerilog and Verilog constructs:
   - Verilog2001 ANSI-C port declarations
   - `always_ff` procedural block
   - `timeunit` and `timeprecision`

## Testing the Counter Design

1. A testbench is provided in the file `counter_test.sv`. Simulate the testbench and counter design.

   You see the following results:

```
time=   0ns clk=1 rst_=x load=x enable=x data=xx count=xx
time=   5ns clk=0 rst_=0 load=x enable=x data=xx count=00
time=  10ns clk=1 rst_=0 load=x enable=x data=xx count=00
time=  15ns clk=0 rst_=1 load=0 enable=1 data=xx count=00
time=  20ns clk=1 rst_=1 load=0 enable=1 data=xx count=01
time=  25ns clk=0 rst_=1 load=0 enable=1 data=xx count=01
time=  30ns clk=1 rst_=1 load=0 enable=1 data=xx count=02
...
time= 105ns clk=0 rst_=1 load=0 enable=1 data=xx count=1e
time= 110ns clk=1 rst_=1 load=0 enable=1 data=xx count=1f
time= 115ns clk=0 rst_=1 load=0 enable=1 data=xx count=1f
time= 120ns clk=1 rst_=1 load=0 enable=1 data=xx count=00
time= 125ns clk=0 rst_=1 load=0 enable=1 data=xx count=00
time= 130ns clk=1 rst_=1 load=0 enable=1 data=xx count=01
COUNTER TEST PASSED
```

   Debug your counter as required.

2. After the test passes, please copy the `counter.sv` file into the `../sv_src` directory. You use it later for the complete VeriRISC design lab.

# Lab 4 Using Classes

**Objective: To use SystemVerilog object-oriented design features.**

Create a simple counter design using the following SystemVerilog object-oriented design features:

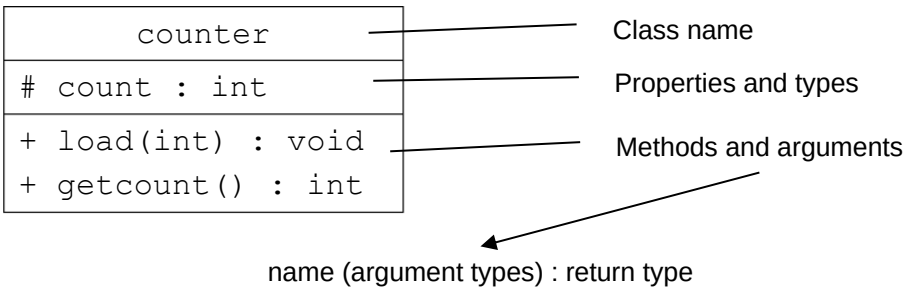
- Class declarations, with explicit constructors and instances
- Inheritance
- Static properties and methods
- Aggregate classes (classes with properties of other class types)

Object-oriented design is complex, and classes are a considerable enhancement to SystemVerilog. If you are having problems with this lab, consult your instructor.

## Creating a Simple Class

Working in the `lab11-countclass` directory, perform the following.

1. Edit the file `counter.sv` to declare a class as described in this diagram.

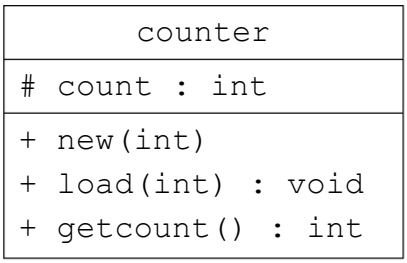



   where `load()` sets property `count` and `getcount ()` returns `count`.

3. Create instances of class `counter` and use the methods. Simulate and debug as needed.

### Adding a Class Constructor

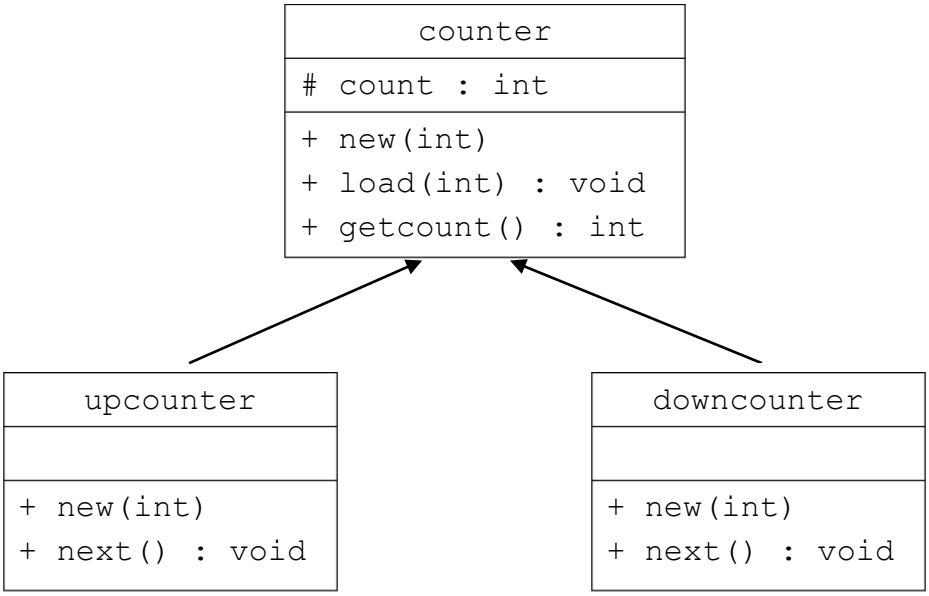
1. Add a class constructor with one `int` argument to set the initial value of `count`. Give the argument a default value of 0.




2. Modify your code to test the constructor. Simulate and debug as needed.

### Defining Derived Classes

1. Extend the `counter` class by declaring two subclasses, `upcounter` and `downcounter`, as follows:




- `upcounter` and `downcounter` inherit the `count` property from `counter`.
- `upcounter` and `downcounter` have explicit constructors, with a single argument, which pass the argument to the `counter` constructor.
- The `upcounter` `next` method increments `count` and display the value for debugging.
- The `downcounter` `next` method decrements `count` and displays the value for debugging.

2. Create instantiations of both subclasses and verify their methods. Simulate and debug as needed.

### Setting Counter Limits

You need to control the upper and lower count limits of both counter subclasses.

1. Add `max` and `min` properties (type `int`) to `counter` for upper and lower count limits.

2. Add a `check_limit` method to `counter` with two input `int` arguments. The method should assign the greater of the two arguments to `max` and the lesser to `min`.

3. Add a `check_set` method to `counter` with a single input `int` argument `set`:
   - If the `set` argument is not within the `max`-`min` limits, assign `count` from `min` and display a warning message. Otherwise, assign `count` from `set`.

4. Add arguments for the `max` and `min` limits to the constructors so they can be set for each class instance. The upcounter and downcounter constructors should pass the limit arguments to the counter constructor.

5. In the `counter` constructor, call `check_limit` with the two limit arguments to ensure that `max` and `min` are consistent and `check_set` to ensure the initial `count` value is within limits.

6. In the `load` method, call `check_set` to ensure that the load value is within limits.

7. Modify the `upcounter` and `downcounter` `next()` methods to count between the two limits, e.g., `upcounter` counts up to `max` and then rolls over to `min`.

8. Modify your test code to check the new functionality. Simulate and debug as needed.

### Indicating Roll-Over and Roll-Under

You need to indicate when an `upcounter` instance rolls over, or `downcounter` instance rolls under.

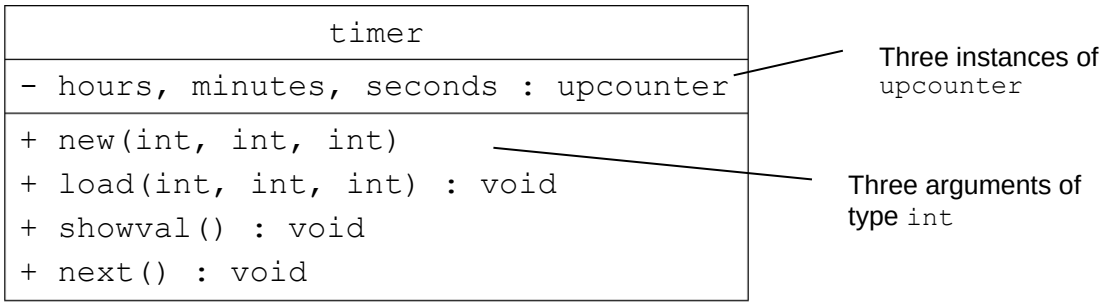
1. Modify `upcounter` as follows:
   a. Add a new property `carry` of type `bit`, initialized to 0 in the constructor.
   b. Modify the `next` method to set `carry` to 1 only when `count` rolls over from `max` to `min`; otherwise, `carry` should be 0.
2. Modify `downcounter` as follows:
   a. Add a new property `borrow` of type `bit`, initialized to 0 in the constructor.
   b. Modify the `next` method to set `borrow` to 1 only when the count rolls under from `min` to `max`, otherwise `borrow` should be 0.
3. Modify your test code to check the `carry` and `borrow` work. Simulate and debug as needed.

### Implementing a Static Property and a Static Method

1. Add a static property to both `upcounter` and `downcounter` to count the number of instances of the class that have been created.
2. Increment the property in the class constructors.
3. Add a static method to both classes, which returns the static property.
4. Modify your test code to check the static properties work. Simulate and debug as needed.
5. For a tool to probe class values, the objects must be constructed (probably not yet at time 0).

## Defining an Aggregate Class

1. Define a new aggregate class `timer` as follows:




where:

- `hours`, `minutes` and `seconds` are three instances of `upcounter`.
- The `timer` constructor has three arguments for the initial hour, minute and second counts (with default values). The constructor creates each `upcounter` instance with the appropriate initial value and sets `max` and `min` to operate the timer as a clock, e.g., the `hours` instance should have a `max` of 23 and a `min` of 0.
- The `load` method has three arguments to set the hour, minute and second counts.
- The `showval` method displays the current hour, minute and second.
- The `next` method increments the timer and displays the new hour, minute and second. Increment the timer by making calls to the `next` methods of the `upcounter` instance and use the `carry` properties to control which `next` method is called, e.g., the `minutes` `next` method is called only when the `seconds` `carry` is 1. Use the `showval` method to display the new values.

Modify your test code to check the timer works. Use `load` to set the timer to values such as `00:00:59` and `next` to check the roll-over. Simulate and debug as needed.